Element Intelligence — Single Source of Truth for Locators

Date: July 5, 2026
Author: System Design Review
Status: ✅ Implemented (Phase 1) — folded into the locator-consistency PRs

---

🎯 Executive Summary

Element Intelligence turns the App Profile into the **single source of truth for locators**.
Every interactive element the crawler sees is distilled into one record: a **semantic name**, a **primary locator**, a set of **confidence-scored, reasoned ranked alternatives**, and a category/role.
Both **Script Generation** and **Self-Healing** now consume the same ranking — so the generator **never invents a locator**, it asks the App Profile, and healing always knows the exact next-best option to try.

The problem this fixes

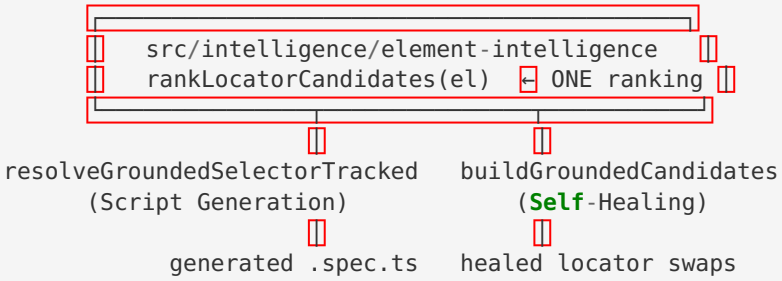
Before this change, LevelUp had **two divergent locator brains**:

Concern	Old Script Generation	Old Self-Healing
Where	`script-gen-engine.ts → resolveGroundedSelector-Tracked`	`app-profile-healing.ts → buildGroundedCandidates`
Shape	single pick	ranked list
Priority	**id-first** (`#user-name`)	**data-test-first** (`[data-test="username"]`)
Output	`{selector, source}`	`AppProfileCandidate[]` `{locator, confidence, reasoning}`

That divergence meant a script could be generated targeting `#login-button` while healing believed the
“correct” locator was `[data-test="login-button"]`. Two systems, two answers, silent drift.

The fix — one canonical brain

A new module, **`src/intelligence/element-intelligence.ts`**, owns the only locator-ranking logic.
Script Generation and Healing both delegate to it.



📊 Part 1: The Canonical Ranking

`rankLocatorCandidates(el, description?)` returns candidates **best-first**, each with a Playwright locator, a raw CSS selector, a strategy, a `0–1` confidence, a human reasoning string, and a `stable` flag. The priority (and confidence) is deliberate and matches how real apps expose automation contracts:

#	Strategy	Example	Confidence	Stable
1	`data-test`	`page.locator('[data-test="username"]')`	0.96	✅
2	`data-testid`	`page.getByTestId('username')`	0.95	✅
3	`data-cy`	`page.locator('[data-cy="username"]')`	0.93	✅
4	`data-qa`	`page.locator('[data-qa="username"]')`	0.92	✅
5	role + accessible name	`page.getByRole('textbox', { name: 'User-name' })`	0.90	✅
6	**stable** id	`page.locator('#user-name')`	0.85	✅
7	name	`page.locator('[name="user-name"]')`	0.83	✅
8	placeholder / label	`page.getByPlaceholder('Username')`	0.80	❌
9	visible text	`page.getByText('Login')`	0.75	❌

Why data-test outranks id (the one design decision)

A `data-test` / `data-testid` attribute is a **dedicated automation contract** — the app author put it there specifically so tests would not break on cosmetic refactors. A raw `id` is often incidental, sometimes framework-generated, and more likely to churn. So for the SauceDemo “Username” field, which
exposes **both** `data-test="username"` and `id="user-name"`, the primary is:

```
1. [data-test="username"]                          96%  dedicated automation contract
2. getByRole('textbox', { name: 'Username' })      90%  resilient to markup changes
3. #user-name                                      85%  stable id
4. [name="user-name"]                              83%  stable for form fields
```

Dynamic ids are never offered. `isDynamicId()` rejects framework hashes (`css-1a2b3c`, `:r0:`, `ember1234`, 8+ hex runs, 4+ digit runs) so we never anchor a test to a value that changes on the next build.

📦 Part 2: The Element Intelligence Record

`buildElementIntelligence(crawlData)` produces one record per addressable element:

```
interface ElementIntelligence {
  semanticName: string;        // "Username", "Login Button"
  role: string;                // "textbox", "button", "link"
  category: string;            // "input", "button", "link", "select"
  primary: LocatorCandidate;   // candidates[0] — what generation & healing use
  candidates: LocatorCandidate[]; // full ranked list (primary included)
  confidence: number;          // primary confidence, surfaced for convenience
  // ---- roadmap metadata (scaffolded now, populated over time) ----
  lastValidated?: string;      // ISO time the element was last seen in a crawl
  usedByScripts?: number;      // how many generated specs reference it
  healedCount?: number;        // how many times healing recovered it
}
```

The `lastValidated` / `usedByScripts` / `healedCount` fields are intentionally scaffolded now so the roadmap features (analytics, “most-healed elements”, staleness detection) can populate them without a
schema change.

Intent resolution

`resolveByIntent(elements, "click the login button")` tokenizes the intent, scores elements by token
overlap (with a role bonus), and returns the winning element’s **ranked candidates** — never manufacturing a match without real token overlap. This is the query API both engines use to answer “give me the best locator for X”.

🔌 Part 3: Integration Points

Script Generation (`resolveGroundedSelectorTracked`)

The old id-first cascade is gone. After the DOM match, generation calls `rankLocatorCandidates(el)` and emits `ranked[0]` — falling back to a computed CSS selector only when
nothing grounds. Generation therefore **never invents or reorders** a locator; the App Profile decides.

The login-triad collapse (`fill(user) + fill(pass) + click(login) → loginPage.login()`) was also made

selector-format-agnostic — it now matches on the semantic token (username/password/login) instead
of a hardcoded `#id`, so page-object reuse keeps working now that fields ground via `data-test`.

Self-Healing (`buildGroundedCandidates`)

Now a thin adapter: it calls `rankLocatorCandidates(el)` and maps the result into `AppProfileCandidate[]` (`source: 'app_profile'`, `validated: true`, grounded reasoning). Healing keeps
its existing top-4 slice and downstream behaviour — but the ranking is now identical to generation's.

Dashboard (Element Intelligence panel)

The Application Profile detail view renders the intelligence directly: per element, the primary locator with a confidence badge and expandable, confidence-scored ranked alternatives (each with its reasoning).
A client-side mirror of `rankLocatorCandidates` keeps the UI in lockstep with the backend ranking, so users **see exactly what the engines resolve**.

✅ Part 4: Testing

- **`tests/unit/element-intelligence.test.ts`** (new) — locks the canonical ranking: data-test outranks
  id, role outranks id for labelled controls, dynamic ids are never offered, `buildElementIntelligence` shape, `resolveByIntent`, `deriveSemanticName`, and empty/edge inputs.
- **`tests/unit/locator-grounding-datatest.test.ts`** & **`multipage-cache-grounding.test.ts`** — updated: dual-hook elements (both `data-test` and `id`) now ground via `data-test`, matching healing.
- **`tests/unit/page-object-reuse.test.ts`**, **`script-gen-scenario-fidelity.test.ts`**, **`script-gen-zip-review-fixes.test.ts`** — pass with the semantic-token login-collapse matcher.
- Full healing suite (`app-profile-healing`, `candidate-ranker`, `healing-advisors`, …) stays green — the ranking output is unchanged for healing; only its source is now shared.

🗺️ Part 5: Roadmap (next phases)

1. **Persist the record** — store `ElementIntelligence` on the App Profile version so `lastValidated`, `usedByScripts`, and `healedCount` accumulate across runs.
2. **Inline editing / pinning** — let users override or pin a primary in the dashboard; generation and healing pick it up automatically (the UI already advertises this).
3. **Analytics** — "most-healed elements", staleness ("not validated in N crawls"), and confidence trend, surfaced in Coverage Intelligence.
4. **RCA / Test Case Lab consumption** — feed the same ranked intelligence into root-cause analysis and
   test-case authoring so every surface speaks one locator language.